UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

Spring WebFlux

Utopios® Tous droits réservés

UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

Fondamentaux réactifs

Utopios® Tous droits réservés

Fondamentaux réactifs

Programmation réactive : concepts et principes

- Comprendre la réactivité
- Reactive Streams & Backpressure
- Patterns essentiels
- Performance vs Servlet

Utopios® Tous droits réservés

Qu'est-ce que la programmation réactive?

La programmation réactive est un **paradigme déclaratif** qui se concentre sur:

- **Le flux de données** : comment les données circulent dans l'application
- **La propagation des changements** : réaction automatique aux changements
- **La composition** : combinaison élégante d'opérations

Utopios® Tous droits réservés

Qu'est-ce que la programmation réactive?

Analogie: Feuille de calcul Excel

Si A1 = 5 et B1 = A1 * 2, alors B1 = 10 automatiquement.

Quand vous changez A1 à 7, B1 devient 14 **instantanément**.

C'est la réactivité: **les changements se propagent**.

Avantages clés

- ✅ Non-bloquant: pas d'attente inutile
- ✅ Scalable: peu de ressources pour beaucoup de requêtes
- ✅ Lisible: code déclaratif et composable
- ✅ Temps réel: réponses rapides et fluides

Utopios® Tous droits réservés

Reactive Streams: la spécification

Reactive Streams définit comment gérer les flux asynchrones avec **non-blocking backpressure**.

Les 4 composants principaux

1. Publisher

Produit les données/événements

Interface: `Publisher<T>`

2. Subscriber

Consomme les données

Interface: `Subscriber<T>`

Utopios® Tous droits réservés

Reactive Streams: la spécification

Les 4 composants principaux

3. Subscription

Lien Publisher-Subscriber

Gère: `request(n)`, `cancel()`

4. Processor

Publisher ET Subscriber

Transforme le flux

Utopios® Tous droits réservés

Reactive Streams: la spécification

Cycle de vie

1. Subscriber s'abonne → `subscribe(subscriber)`
2. Publisher appelle → `onSubscribe(subscription)`
3. Subscriber demande → `subscription.request(10)`
4. Publisher envoie → `onNext(element)` (max 10)
5. Fin du flux → `onComplete()` ou `onError(exception)`

Utopios® Tous droits réservés

Backpressure: Le concept clé

La **backpressure** est le mécanisme qui permet au Subscriber de dire au Publisher: *"Je ne peux traiter que X éléments par seconde"*

Problème sans backpressure

❌ **Sans contrôle:** Publisher envoie 1 million d'éléments instantanément

→ Arrêt mémoire (OutOfMemoryError)

→ Application crash

Utopios® Tous droits réservés

Backpressure: Le concept clé

Solution avec backpressure

✅ **Avec backpressure:** Subscriber dit "request(10)" au Publisher

→ Publisher envoie exactement 10 éléments

→ Subscriber les traite

→ Reclame 10 de plus quand prêt

→ Équilibre parfait!

Utopios® Tous droits réservés

Backpressure: Le concept clé

Code illustratif

```
// Subscriber demande du débit
subscription.request(10);  // "J'en veux 10"

// Publisher envoie max 10
onNext(element1);
onNext(element2);
// ...
onNext(element10);

// Subscriber les traite
// Puis demande plus
subscription.request(10);  // "10 de plus!"
```

Utopios® Tous droits réservés

Patterns réactifs essentiels

1. Map - Transformer chaque élément

```
flux.map(x -> x * 2)  // 1→2, 2→4, 3→6, ...
```

2. Filter - Garder certains éléments

```
flux.filter(x -> x % 2 == 0)  // Seulement les pairs
```

Utopios® Tous droits réservés

Patterns réactifs essentiels

3. FlatMap - Aplatir les flux imbriqués

```
flux.flatMap(user ->
    getUserOrders(user)  // Retourne Flux<Order>
)  // Résultat: Flux<Order> aplati
```

4. Merge - Combiner plusieurs flux

```
Flux.merge(
    activeUsers,
    inactiveUsers
)  // Tous les utilisateurs mélangés
```

Utopios® Tous droits réservés

Patterns réactifs essentiels

5. Zip - Synchroniser deux flux

```
Flux.zip(users, orders)
// Paire [User, Order] quand les deux sont prêts
```

💡 **Ces patterns sont le fondement de tout ce qu'on fait avec Project Reactor!**

Utopios® Tous droits réservés

Trois paradigmes comparés

1. Synchrone (Bloquant) ❌

```
UserDto user = userService.getUser(1);
// ⏸️ ON ATTEND ICI - thread complètement bloqué
System.out.println(user.getName());
```

2. Asynchrone (Callback Hell) 😞

```
userService.getUser(1, user -> {
    userService.getOrders(user.getId(), orders -> {
        orders.forEach(order -> {
            System.out.println(order);
        });
    });
});
```

Utopios® Tous droits réservés

Trois paradigmes comparés

3. Réactif (Déclaratif) ✨

```
userService.getUser(1)
    .flatMap(user -> userService.getOrders(user.getId()))
    .doOnNext(order -> System.out.println(order))
    .subscribe();
```

La réactivité = lisibilité + efficacité!

Utopios® Tous droits réservés

Performance: Servlet vs WebFlux

Comparaison pour 1000 requêtes simultanées

Servlet (Bloquant)

- 1000 threads créés
- ~1MB par thread
- Total: ~1GB RAM
- Changements contexte coûteux

Utopios® Tous droits réservés

Performance: Servlet vs WebFlux

Comparaison pour 1000 requêtes simultanées

WebFlux (Réactif)

- 10-20 threads max
- ~1MB par thread
- Total: ~20MB RAM
- Pas de changement contexte

Utopios® Tous droits réservés

Performance: Servlet vs WebFlux

Comparaison pour 1000 requêtes simultanées

Idéal pour WebFlux

I/O Non-bloquant, Haute Concurrence, Peu de CPU

Pas idéal pour WebFlux

CPU Intensif, Blocs longs

💡 **Verdict:** WebFlux excelle quand vous avez beaucoup de requêtes I/O (API calls, BD) avec peu de traitement CPU.

Utopios® Tous droits réservés

Résumé: Points clés à retenir

✅ Concepts essentiels

- **Programmation réactive** = déclarative + orientée événements
- **Reactive Streams** = Publisher/Subscriber avec backpressure
- **Backpressure** = Subscriber contrôle le débit des données
- **Non-bloquant** = meilleure utilisation des ressources
- **Patterns** = map/filter/flatMap/merge/zip = fondamentaux
- **WebFlux** excelle avec beaucoup d'I/O concurrente

Utopios® Tous droits réservés

UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

Project Reactor

Utopios® Tous droits réservés

Project Reactor

Mono, Flux et opérateurs essentiels

- Mono vs Flux
- Opérateurs transformation
- Combinaison de flux
- Gestion erreurs

Utopios® Tous droits réservés

Mono vs Flux: Quand utiliser quoi?

Project Reactor fournit deux implémentations principales de Publisher:

Mono: Zéro ou UN élément

Cas d'usage: Une seule valeur (ou rien)

- Requête HTTP GET par ID
- Opération de base de données uniq
- Calcul unique

```
Mono<User> user = Mono.just(new User(1, "Alice"));
Mono<Void> deleted = Mono.empty();
Mono<String> delayed = Mono.delay(Duration.ofSeconds(1));
```

Utopios® Tous droits réservés

Mono vs Flux: Quand utiliser quoi?

Flux: Zéro à INFINI d'éléments

Cas d'usage: Une séquence de valeurs

- Récupérer tous les utilisateurs
- Flux de tweets ou événements
- WebSocket ou SSE (streaming)

```
Flux<Integer> numbers = Flux.range(1, 5);  // 1,2,3,4,5
Flux<String> items = Flux.just("a", "b", "c");
Flux<Long> interval = Flux.interval(Duration.ofSeconds(1));
```

Utopios® Tous droits réservés

Opérateurs de transformation

map: Transformer chaque élément

```
Flux.just(1, 2, 3)
    .map(x -> x * 2)
    .subscribe(System.out::println);  // Affiche: 2, 4, 6
```

filter: Garder seulement certains éléments

```
Flux.range(1, 10)
    .filter(x -> x % 2 == 0)
    .subscribe(System.out::println);  // Affiche: 2, 4, 6, 8, 10
```

Utopios® Tous droits réservés

Opérateurs de transformation

flatMap: Aplatir les flux imbriqués

```
Flux.just(1, 2, 3)
    .flatMap(userId -> fetchUserDetails(userId))
    .subscribe(System.out::println);  // Détails mélangés
```

concatMap: Aplatir en préservant l'ordre

```
Flux.just(1, 2, 3)
    .concatMap(userId -> fetchUserDetails(userId))
    .subscribe(System.out::println);  // 1 puis 2 puis 3
```

Utopios® Tous droits réservés

Opérateurs de transformation

distinct, take, skip

```
Flux.just(1, 2, 2, 3, 3, 3)
    .distinct()
    .subscribe();  // 1, 2, 3

Flux.range(1, 100)
    .take(5)
    .subscribe();  // 1, 2, 3, 4, 5
```

Utopios® Tous droits réservés

Combinaison de flux

merge: Combiner sans ordre garanti (rapide)

```
Flux<Integer> flux1 = Flux.just(1, 2);
Flux<Integer> flux2 = Flux.just(10, 20);
Flux.merge(flux1, flux2)
    .subscribe();  // 1, 10, 2, 20 (possiblement)
```

concat: Combiner séquentiellement (ordre préservé)

```
Flux.concat(flux1, flux2)
    .subscribe();  // 1, 2, 10, 20
```

Utopios® Tous droits réservés

Combinaison de flux

zip: Synchroniser deux flux en paires

```
Flux<String> names = Flux.just("Alice", "Bob");
Flux<Integer> ages = Flux.just(30, 25);
Flux.zip(names, ages)
    .map(t -> t.getT1() + " age " + t.getT2())
    .subscribe();  // Alice age 30, Bob age 25
```

combineLatest: Combiner les dernières valeurs

```
Flux.combineLatest(userFlux, orderFlux)
    .map((user, orders) -> new UserWithOrders(user, orders))
    .subscribe();
```

Résumé: merge=rapide, concat=séquentiel, zip=synchronisé, combineLatest=réactif

Utopios® Tous droits réservés

Gestion des erreurs

onErrorReturn: Retourner une valeur par défaut

```
fetchUser(999)
    .onErrorReturn(new User(0, "Anonymous"))
    .subscribe();  // Affiche User(0, "Anonymous")
```

onErrorResume: Basculer vers un autre flux

```
fetchUser(999)
    .onErrorResume(error ->
        Mono.just(new User(0, "Default User"))
    )
    .subscribe();
```

Utopios® Tous droits réservés

Gestion des erreurs

retry: Réessayer automatiquement

```
unstableApi()
    .retry(3)  // Réessai jusqu'à 3 fois
    .subscribe(
        System.out::println,
        error -> System.err.println("Échec après 3 tentatives")
    );
```

Utopios® Tous droits réservés

Gestion des erreurs

timeout: Arrêter après un délai

```
slowOperation()
    .timeout(Duration.ofSeconds(5))
    .onErrorResume(error ->
        Mono.just("Timeout, retour défaut")
    )
    .subscribe();
```

⚠️ Attention: Si vous ne gérez pas les erreurs, le flux meurt silencieusement! Toujours penser au onError*.

Utopios® Tous droits réservés

Subscription et hooks (doOn*)

Lifecycle complet d'une subscription

```
Mono.just("Hello")
    .doOnSubscribe(s -> System.out.println("Abonné!"))
    .doOnNext(v -> System.out.println("Émettre: " + v))
    .doOnError(e -> System.err.println("Erreur: " + e))
    .doOnComplete(() -> System.out.println("Terminé!"))
    .doFinally(t -> System.out.println("Cleanup"))
    .subscribe();
```

Utopios® Tous droits réservés

Subscription et hooks (doOn*)

Les 3 formes de subscribe()

```
// Forme 1: Aucun comportement (flux lancé mais ignoré!)
flux.subscribe();

// Forme 2: Avec traitement des éléments
flux.subscribe(value -> System.out.println(value));

// Forme 3: Avec gestion complète
flux.subscribe(
    value -> System.out.println("Element: " + value),
    error -> System.err.println("Erreur: " + error),
    () -> System.out.println("Complété!")
);
```

💡 Pro tip: Rien ne se passe jusqu'à `subscribe()`! C'est "lazy evaluation". Le flux ne s'exécute que quand quelqu'un s'abonne.

Utopios® Tous droits réservés

Opérateurs de temps

delay: Délai avant d'émettre

```
Mono.just("Bonjour")
    .delayElement(Duration.ofSeconds(2))
    .subscribe();  // Affiche après 2 secondes
```

interval: Émettre à intervalle régulier

```
Flux.interval(Duration.ofSeconds(1))
    .take(5)
    .subscribe();  // 0, 1, 2, 3, 4 avec 1s entre
```

Utopios® Tous droits réservés

Opérateurs de temps

debounce: Ignorer les émissions rapides

```
userInput  // Flux de frappes utilisateur
    .debounce(Duration.ofMillis(300))
    .flatMap(query -> search(query))
    .subscribe();  // Cherche seulement après 300ms inactivité
```

buffer & window

```
userIds
    .buffer(100)  // Lots de 100
    .subscribe();  // [1-100], [101-200], ...

flux.window(Duration.ofSeconds(1))  // Fenêtres temporelles
    .subscribe();
```

Utopios® Tous droits réservés

Schedulers et threading

Où s'exécute le code?

Par défaut, Project Reactor s'exécute sur le **thread appelant**. Pour contrôler:

publishOn: Changer le contexte d'exécution

```
fetchData()  // Thread principal
    .map(data -> data * 2)  // Même thread
    .publishOn(Schedulers.boundedElastic())
    .map(data -> doSlowOperation(data))  // Thread du pool
    .subscribe();
```

Utopios® Tous droits réservés

Schedulers et threading

subscribeOn: Exécuter la subscription sur un thread

```
blockingAPI()
    .subscribeOn(Schedulers.boundedElastic())
    .subscribe();  // Blockage sur thread pool, pas principal
```

Utopios® Tous droits réservés

Schedulers et threading

Schedulers disponibles

- **boundedElastic()**
  - I/O bloquants
  - Pool dynamique
- **parallel()**
  - CPU-intensive
  - CPUs
- **single()**
  - Séquentiel
  - 1 thread

🎯 Règle d'or: `boundedElastic()` pour I/O, `parallel()` pour CPU.

Utopios® Tous droits réservés

Résumé: Points clés

✅ À retenir

- **Mono** = 0 ou 1 élément (single value)
- **Flux** = 0 ou N éléments (sequences)
- **Lazy** = rien n'exécute jusqu'à `subscribe()`
- **Immutable** = chaque opération retourne un nouveau flux
- **Chainable** = composez facilement avec `.then()`
- **Error handling** = `onError*` ou gestion dans subscribe

Opérateurs clés résumés

map, filter, flatMap, concatMap, merge, zip, retry, timeout

Utopios® Tous droits réservés

Bonus: Patterns avancés

collectList: Flux → Mono

```
userFlux
    .collectList()
    .subscribe(users -> System.out.println("Total: " + users.size()));
```

scan: Accumuler les valeurs

```
Flux.just(1, 2, 3, 4, 5)
    .scan(0, (acc, x) -> acc + x)
    .subscribe();  // 0, 1, 3, 6, 10, 15
```

Utopios® Tous droits réservés

Bonus: Patterns avancés

switchIfEmpty: Fournir une alternative

```
userService.getUserById(id)
    .switchIfEmpty(Mono.just(new User(0, "Default")))
    .subscribe();
```

repeat: Répéter le flux

```
Mono.just("Retry")
    .repeat(3)  // Répète 3 fois
    .subscribe();
```

💡 Ces patterns vous seront utiles dans des cas avancés!

Utopios® Tous droits réservés

UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

Merci pour votre attention

Des questions ?

Utopios® Tous droits réservés